Rising Waters - Flood Prediction System Project

Project Overview

Project Title
Rising Waters - Flood Prediction System

Team Members
- Al Rihab Chandhini Mohammed (Team Leader)
- Ranadeep Vinukonda
- Prem Satya Sai Udatha
- Satyanarayana Akula
- Vijay Kiran Kommoju

Project Description
Rising Waters is an AI-powered web application that predicts the possibility of floods using historical weather and rainfall data. The system applies machine learning algorithms to analyze environmental conditions and classify whether a flood is likely to occur.
The project helps disaster management authorities, researchers, and the public make informed decisions through early flood prediction.

Problem Statement
Floods are one of the most devastating natural disasters, causing loss of life, property damage, and economic disruption. Existing warning systems often require significant manual monitoring or expensive infrastructure.
There is a need for a simple, intelligent system capable of predicting flood risk based on historical weather information.

Proposed Solution
Develop an AI-powered flood prediction application using Machine Learning.
The system:
- Collects weather parameters
- Processes the input
- Predicts flood occurrence
- Displays results instantly through a Flask web application

Scenarios

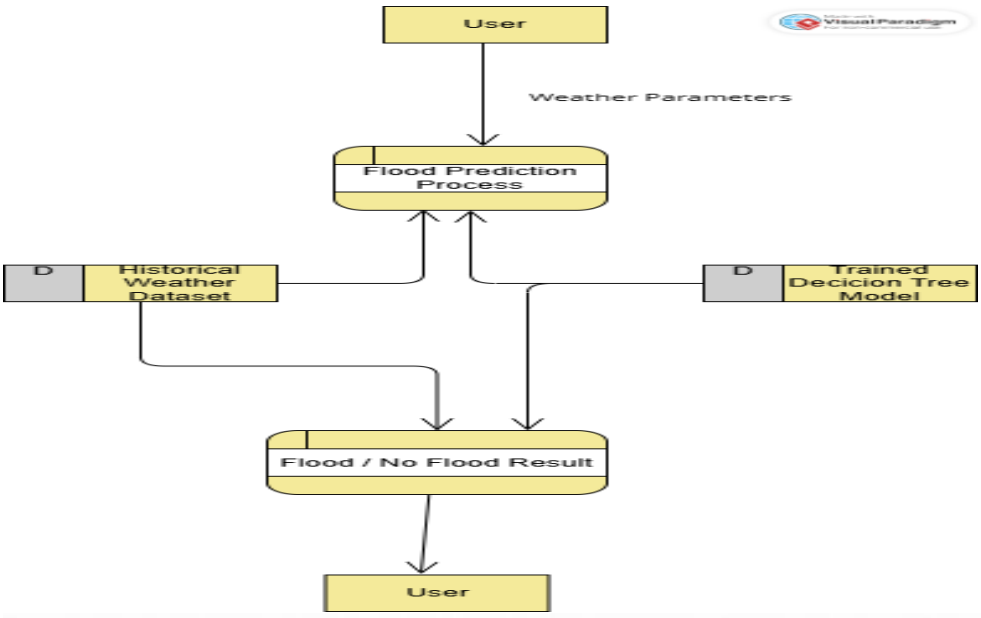
Scenario 1:
A disaster management officer enters weather and rainfall data into the system. The application predicts whether a flood is likely to occur, helping authorities take timely preventive measures.

Scenario 2:

A farmer enters seasonal weather conditions before planting crops. The system predicts flood risk, helping the farmer plan cultivation and reduce potential losses.

Scenario 3:
A researcher inputs historical weather data to analyze flood-prone conditions. The system provides predictions that support flood risk analysis and future planning.

Technical Architecture:



Description:
The system accepts weather parameters from the user, processes them through the Flask backend using the trained machine learning model, and predicts whether flood conditions are likely. The prediction result is then displayed to the user through the web application.
(Note: If you are using database in your project definitely you need to add ER Diagram along with description)

Pre-requisites

- **Flask Framework Knowledge:** Flask Documentation
- **Machine Learning Fundamentals:** Scikit-learn Documentation
- **Python Programming Proficiency:** Python Documentation
- **HTML, CSS, and JavaScript Skills:** W3Schools HTML/CSS/JavaScript Tutorials
- **Data Analysis Libraries:** Pandas and NumPy Documentation
- **Data Visualization Basics:** Matplotlib Documentation
- **Version Control with Git:** Git Documentation
- **Development Environment Setup:** Flask Installation Guide

Project Workflow

Milestone 1: Dataset Collection & System Architecture

Activity 1.1: Collect Historical Weather Dataset

- Download historical weather and rainfall datasets from Kaggle.
- Verify dataset quality and remove duplicate records.

Activity 1.2: Data Preprocessing

- Handle missing values.
- Convert categorical data.
- Normalize numerical values.
- Prepare the dataset for machine learning.

Activity 1.3: Define System Architecture

- Design the architecture showing Frontend, Flask Backend, Machine Learning Model, and Dataset.
- Define data flow between components.

Activity 1.4: Development Environment Setup

- Install Python.
- Install Flask.
- Install required libraries.
- Configure the project structure.

Milestone 2: Core Functionalities Development

Activity 2.1: Develop the Core Functionalities

Develop the flood prediction module using a trained Machine Learning model.

Implement weather data preprocessing and feature handling.

Generate real-time Flood or No Flood predictions based on user inputs.

Display prediction results through a user-friendly web interface.

Activity 2.2: Implement the Flask Backend

Develop the Flask backend to manage application routing and process user inputs.

Load the trained machine learning model (model.pkl) for prediction.

Validate user input data and pass it to the prediction model.

Return prediction results dynamically to the frontend for display.

Milestone 3: App.py Development

Activity 3.1: Develop the main application logic in app.py, create Flask routes, process user inputs, load the trained machine learning model, and display the flood prediction results.

Milestone 4: Frontend Development

- **Activity 4.1:** Design and develop a responsive user interface using **HTML, CSS, and JavaScript** for entering weather parameters and displaying prediction results.
- **Activity 4.2:** Use **Flask's render_template()** to render web pages and dynamically display **Flood** or **No Flood** prediction results based on user input.

Milestone 5: Deployment

- **Activity 5.1:** Prepare the application for local deployment by configuring the Flask environment and installing all required dependencies.
- **Activity 5.2:** Run the application locally, verify all functionalities, and test the flood prediction system using sample weather data.

Milestone 6: Conclusion

Milestone 1: Dataset Preparation and System Architecture

In this milestone, historical weather and rainfall data were collected and preprocessed. Multiple machine learning models were trained and evaluated to select the best model for flood prediction. The system architecture was designed, and the development environment was configured.

Activity 1.1: Collect and Prepare the Dataset

- Collect the dataset from Kaggle.
- Clean and preprocess the data.
- Split the dataset into training and testing sets.

Activity 1.2: Select and Train Machine Learning Models

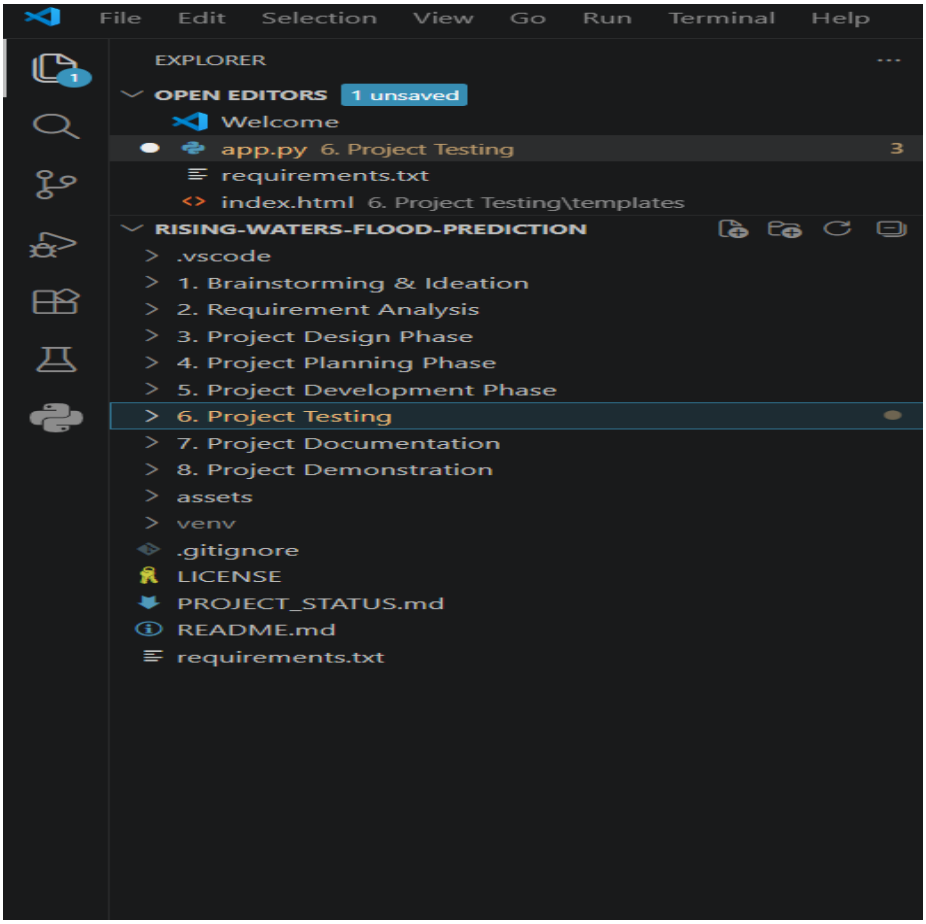
- Train Decision Tree, Random Forest, KNN, and XGBoost models.
- Compare model performance and save the best model.

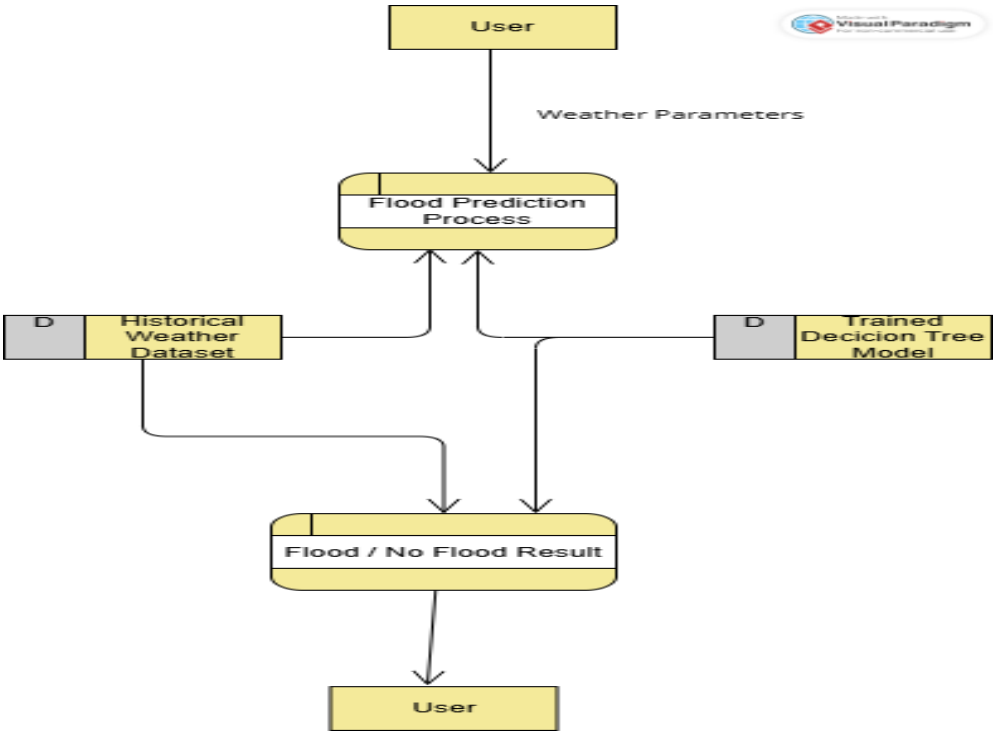
Activity 1.3: Define the System Architecture

- Design the architecture showing the Frontend, Flask Backend, Machine Learning Model, and Dataset.
- Define the data flow for flood prediction.

Activity 1.4: Set Up the Development Environment

- Install Python, Flask, and required libraries.
- Configure the project structure and dependencies.


File Edit Selection View Go Run Terminal Help
EXPLORER
OPEN EDITORS 1 unsaved
Welcome
app.py 6. Project Testing 3
requirements.txt
index.html 6. Project Testing\templates
RISING-WATERS-FLOOD-PREDICTION
.vscode
1. Brainstorming & Ideation
2. Requirement Analysis
3. Project Design Phase
4. Project Planning Phase
5. Project Development Phase
6. Project Testing
7. Project Documentation
8. Project Demonstration
assets
venv
.gitignore
LICENSE
PROJECT_STATUS.md
README.md
requirements.txt


User
Weather Parameters
Flood Prediction Process
D
Historical Weather Dataset
D
Trained Decicion Tree Model
Flood / No Flood Result
User

Milestone 2: Core Functionalities Development

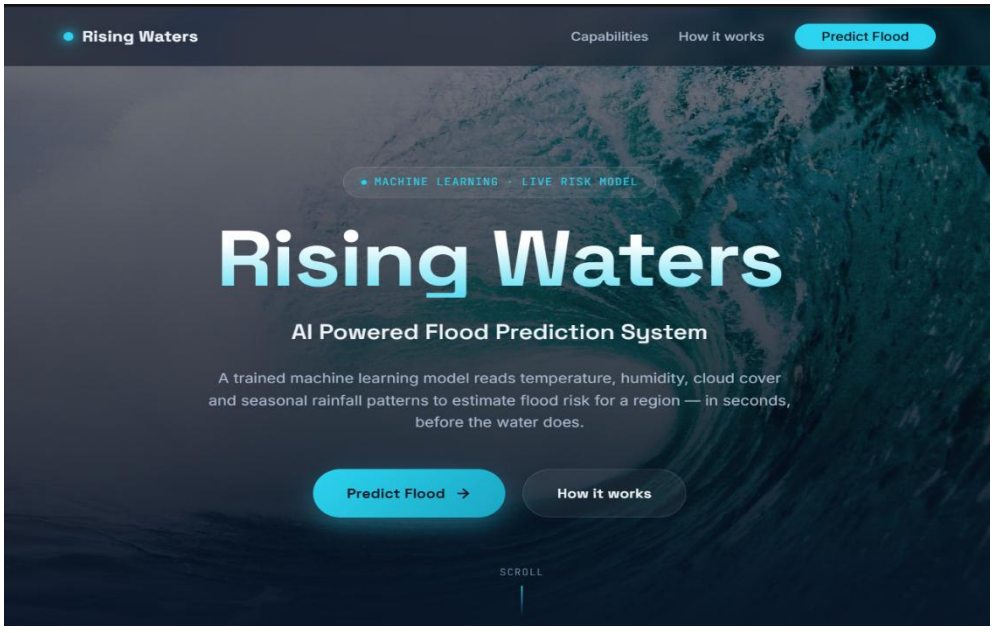
This milestone focuses on implementing the main functionalities of the Flood Prediction System using Machine Learning and Flask.

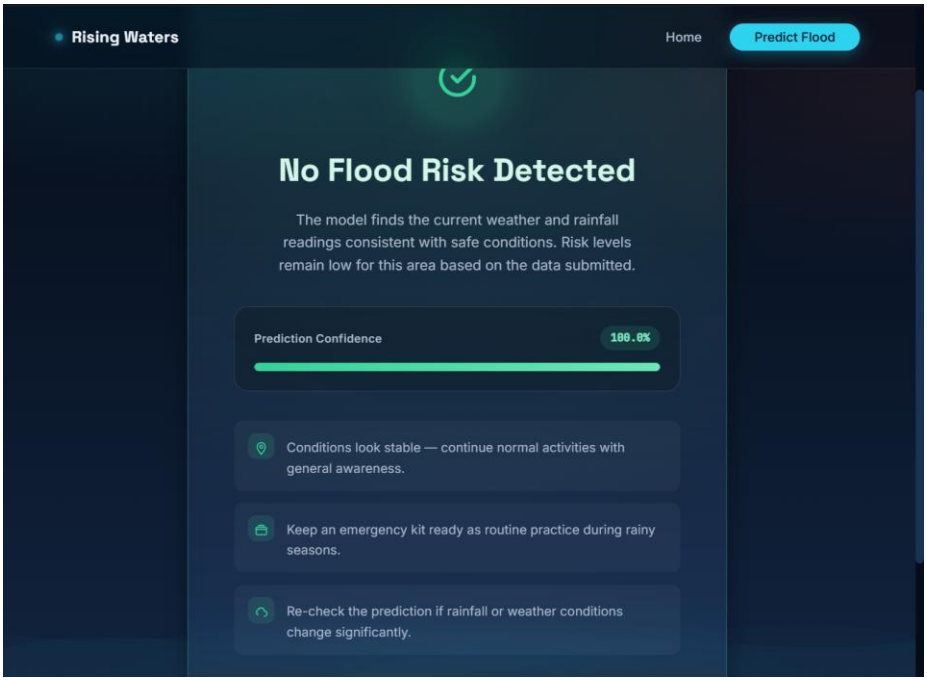
Activity 2.1: Develop Core Functionalities

- Implement data preprocessing and feature handling.
- Train multiple machine learning models for flood prediction.
- Predict whether flooding is likely based on weather inputs.
- Display prediction results through the web application.

Activity 2.2: Implement the Flask Backend

- Create Flask routes for the home page and prediction page.
- Process user inputs and validate the data.
- Load the trained model and generate flood prediction results.
- Return the prediction to the frontend for display.





Milestone 3: App.py Development

This milestone focuses on developing the main application logic using Flask.

Activity 3.1: Develop the Main Application Logic

- Create Flask routes for the application.
- Load the trained machine learning model.
- Accept weather parameters from the user.
- Process the input data and generate flood predictions.
- Display the prediction results and handle invalid inputs.

```
    "scaler.pkl"
)

model = joblib.load(MODEL_PATH)
scaler = joblib.load(SCALER_PATH)

# ------------------------------------------------
# Routes
# ------------------------------------------------

@app.route("/")
def home():
    return render_template("home.html")


@app.route("/predict")
def predict_page():
    return render_template("index.html")


@app.route("/chance")
def chance():
    return render_template("chance.html")


@app.route("/no_chance")
def no_chance():
    return render_template("no_chance.html")


# ------------------------------------------------
# Prediction
# ------------------------------------------------
```

Milestone 4: Frontend Development

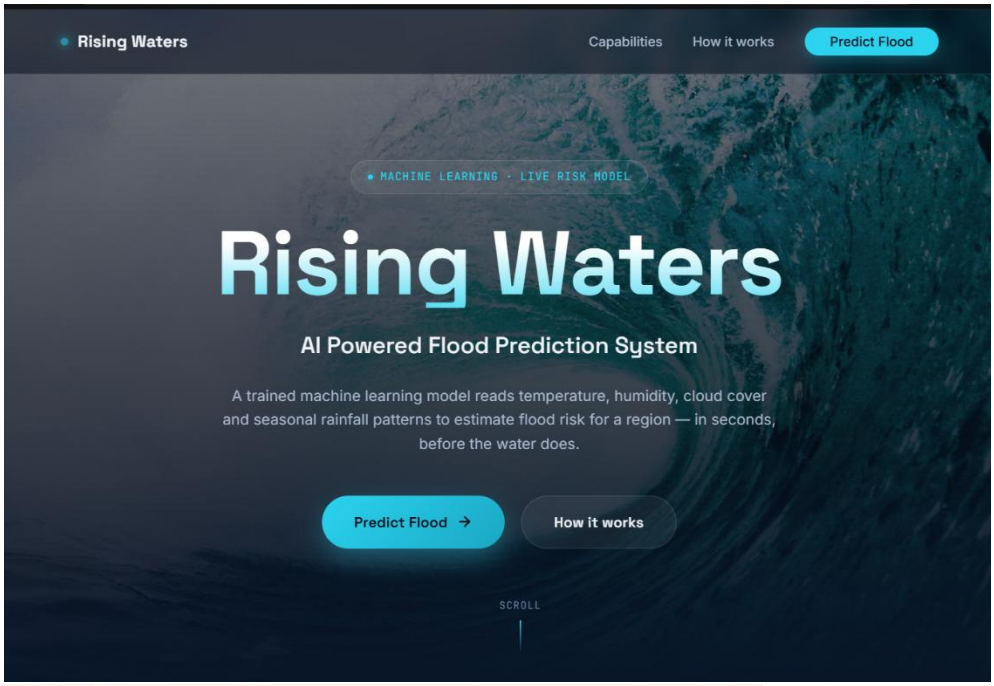
This milestone focuses on developing a responsive and user-friendly interface for the Flood Prediction System using HTML, CSS, JavaScript, and Flask.

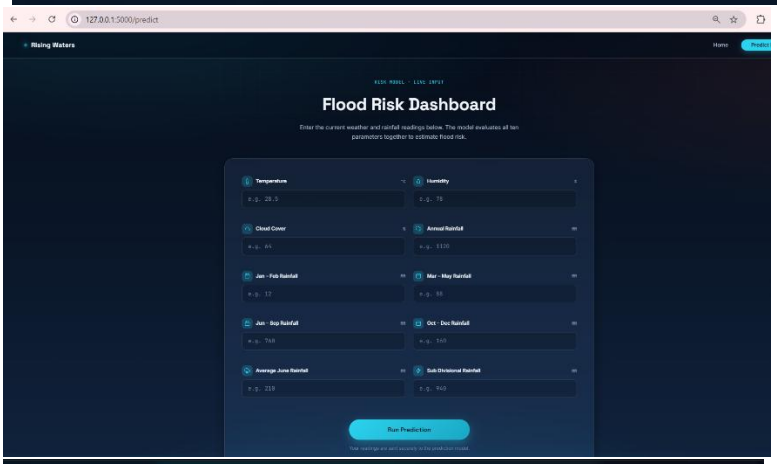
Activity 4.1: Design and Develop the User Interface

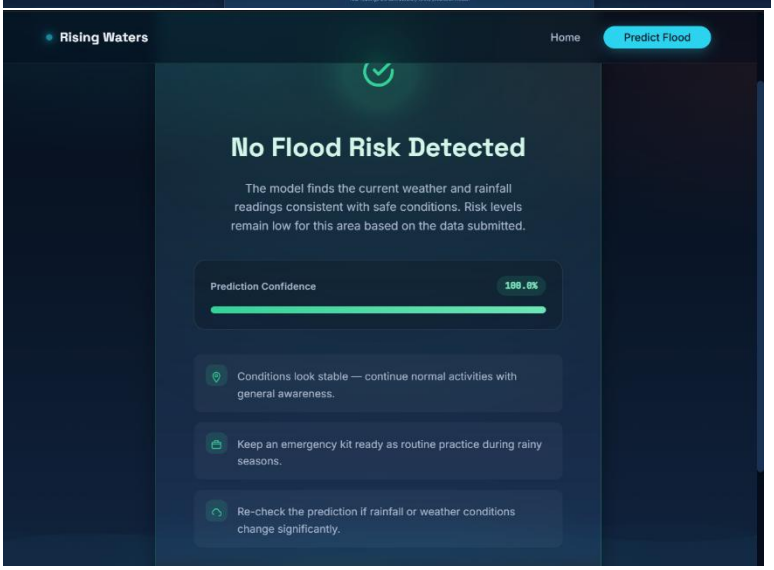
- Develop the home page using HTML and CSS.
- Create input fields for weather and rainfall parameters.
- Design a responsive and easy-to-use interface.
- Display prediction results clearly to the user.

Activity 4.2: Integrate Frontend with Flask

- Use Flask's render_template() to render web pages.
- Capture user inputs and send them to the Flask backend.
- Display **Flood** or **No Flood** prediction results dynamically.
- Validate user inputs and handle invalid entries.


Rising Waters
Capabilities
How it works
Predict Flood
MACHINE LEARNING · LIVE RISK MODEL
Rising Waters
AI Powered Flood Prediction System
A trained machine learning model reads temperature, humidity, cloud cover and seasonal rainfall patterns to estimate flood risk for a region — in seconds, before the water does.
Predict Flood →
How it works
SCROLL


Flood Risk Dashboard


Rising Waters
Home
Predict Flood
No Flood Risk Detected
The model finds the current weather and rainfall readings consistent with safe conditions. Risk levels remain low for this area based on the data submitted.
Prediction Confidence
100.0%
Conditions look stable — continue normal activities with general awareness.
Keep an emergency kit ready as routine practice during rainy seasons.
Re-check the prediction if rainfall or weather conditions change significantly.

Milestone 5: Deployment

This milestone focuses on preparing the Flood Prediction System for local execution and verifying that all functionalities work correctly.

Activity 5.1: Prepare the Application for Local Deployment

- Install Python and the required libraries from requirements.txt.
- Configure the Flask environment.
- Organize the project files and dependencies.

Activity 5.2: Local Testing and Verification

- Run the application using python app.py.
- Access the application through the local Flask server.
- Test the system with different weather and rainfall inputs.
- Verify that the application correctly predicts **Flood** or **No Flood**.

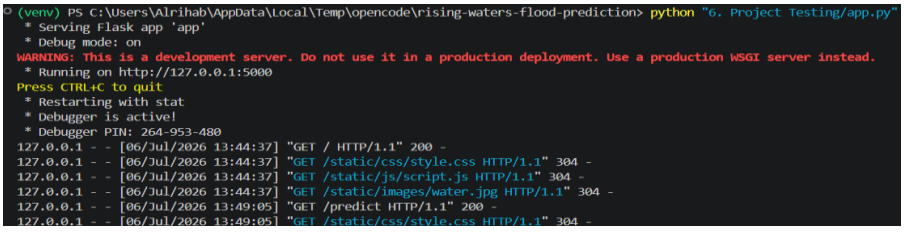
Run Commands

python -m venv myenv

myenv\Scripts\activate

pip install -r requirements.txt

python app.py



Open the application in your browser:

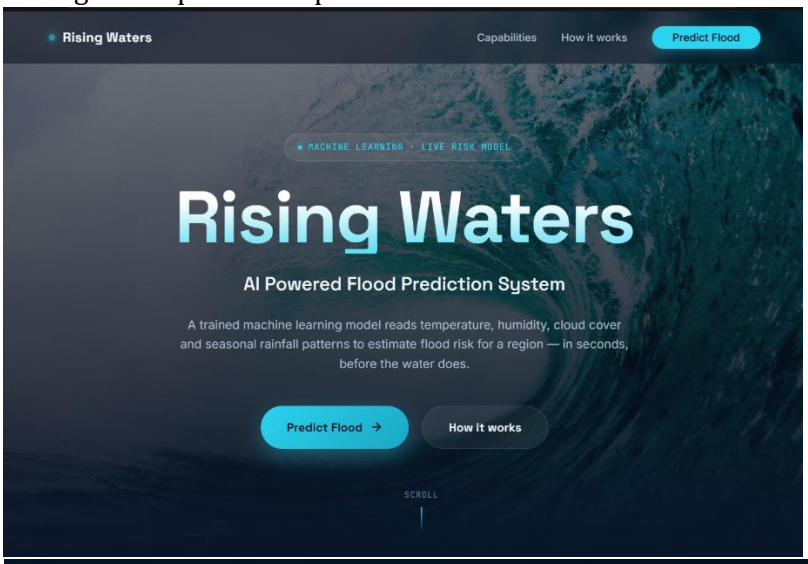
http://127.0.0.1:5000

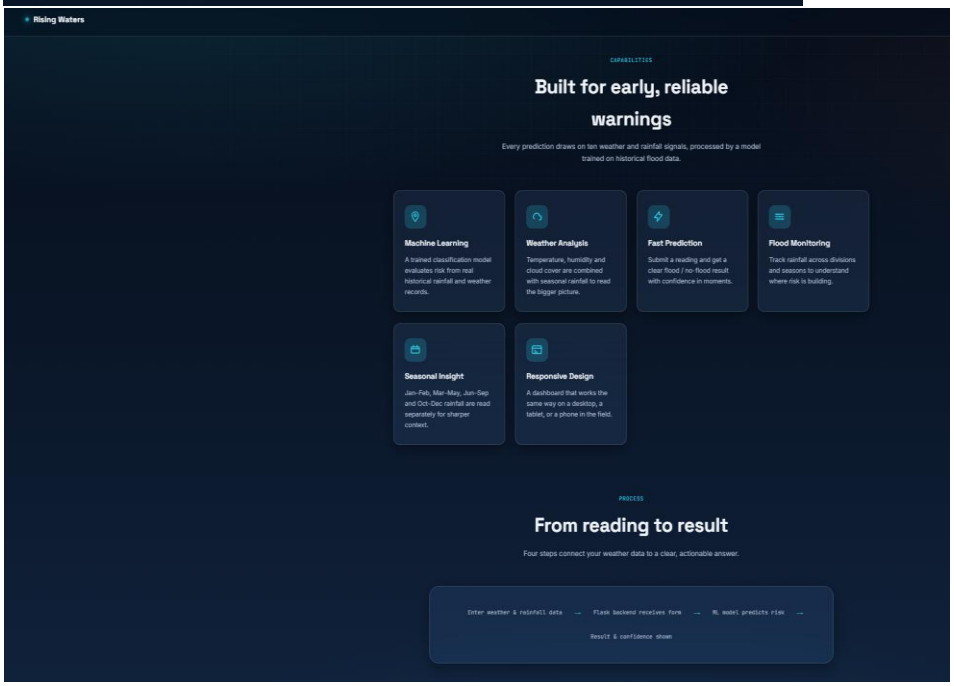
Exploring the Web Pages

1. Home Page

Description:

The home page allows users to enter weather and rainfall parameters required for flood prediction through a simple and responsive interface.

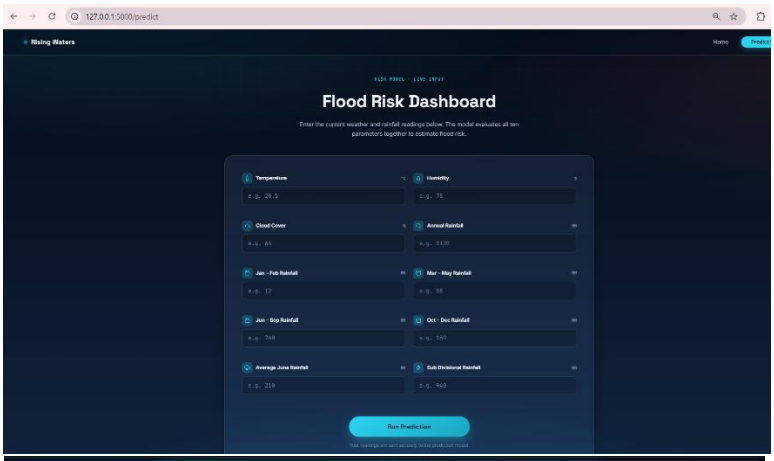


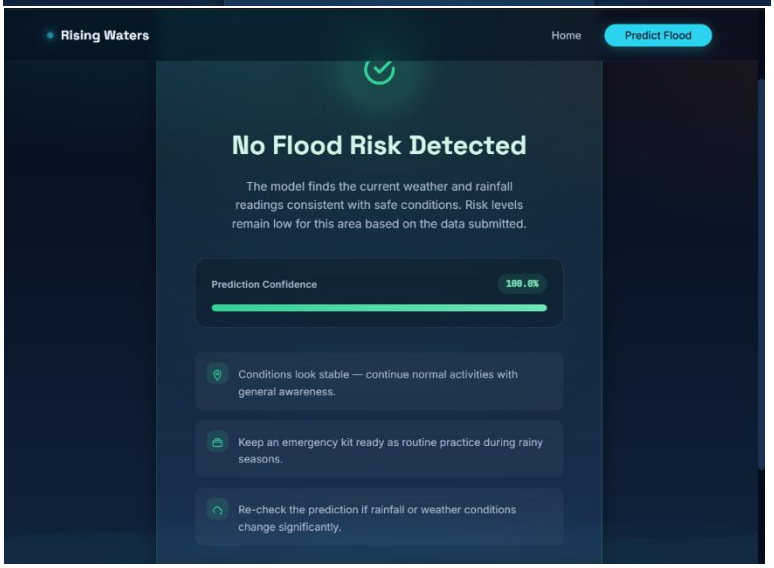


2. Prediction Result Page

Description:

After submitting the input values, the system processes the data using the trained machine learning model and displays the prediction result as Flood or No Flood.





Conclusion

The **Rising Waters - Flood Prediction System** is a machine learning-based web application developed using **Flask, Python, HTML, CSS, and JavaScript** to predict the likelihood of floods based on weather and rainfall parameters. Multiple machine learning algorithms were evaluated, and the best-performing model was integrated into the application to provide accurate predictions through a simple and user-friendly interface. The project demonstrates how machine learning can support early flood prediction and disaster preparedness. In the future, the system can be enhanced by integrating real-time weather APIs, cloud deployment, user authentication, prediction history, and interactive maps to improve its accuracy, scalability, and practical usability.